Comprehensive Technical Report: Dynamic Modeling, State-Space Linearization, and PID Control of a Self-Balancing Two-Wheeled Robot

Department of Electrical Engineering and Computer Science
Indian Institute of Science Education and Research (IISER) Bhopal

Project Guide: Dr. Arijit Sen

Scribed by: Nikhil Rathore, Balam Harish

Abstract

The two-wheeled self-balancing robot represents a canonical problem in modern control theory, functioning as a highly non-linear, underactuated inverted pendulum. Without continuous active control, the system is inherently unstable and will succumb to gravitational forces. This report details the comprehensive mechatronic design, strict mathematical modeling via Lagrangian and Newtonian mechanics, system linearization through Jacobian matrices, noise attenuation via Kalman filtering, and the implementation of a Proportional-Integral-Derivative (PID) closed-loop controller. The analytical frameworks presented herein validate the capability of linear control systems to stabilize non-linear robotic architectures around a defined operational equilibrium. Furthermore, extensive MATLAB and Simulink simulations validate the theoretical model, demonstrating successful state-space extraction, step-response stabilization, and dynamic physical animation. Finally, a 3D rigid-body physics simulation was constructed using CoppeliaSim to validate the real-world behavior of the robotic system.

Contents

1 Introduction and Problem Statement **3**

2 Hardware and System Architecture **3**
2.1 System Parameters 3

3 Dynamic System Modeling **4**
3.1 Electromechanical Actuator (DC Motor) Model 4
3.2 Mechanical Chassis Dynamics 4

4 Linearization and State-Space Formulation **5**
4.1 General Linearization Principle (Taylor Series) 5
4.2 Small-Angle Approximations Application 5
4.3 State-Space Representation 6

5 Sensor Fusion and State Estimation (Kalman Filter) **6**

6 Control Strategy and Simulation Results **7**
6.1 Numerical State-Space and Transfer Function 7
6.2 PID Control and Tuning Methodology 7
6.3 Simulink Block Diagram Setup 8
6.4 Step Response and Scope Analysis 9

7 Conclusion **11**

8 Future Scope **11**

9 3D Rigid Body Simulation in CoppeliaSim **12**
9.1 Model Construction and Hierarchy 12
9.2 Troubleshooting and Dynamic Properties 13
9.3 Simulation Resolution and Balancing Performance 14

A MATLAB Source Code **16**
A.1 System Calculation and Step Response Script 16
A.2 Robot Animation Script 17

1 Introduction and Problem Statement

The self-balancing two-wheeled robot has gained immense popularity in both academic research and commercial applications (e.g., the Segway personal transporter). It serves as a benchmark for testing advanced control algorithms due to its complex, coupled dynamics.

Problem Statement: The fundamental problem lies in the system's underactuated nature; it possesses more degrees of freedom (translation across the floor and pitching forward/backward) than it has independent actuators (the two wheels). If the robot deviates even slightly from the vertical axis, gravity exerts an exponentially increasing destabilizing torque. To maintain an upright position, a closed-loop control system must measure the tilt, compute the necessary corrective force, and apply torque to the wheels to drive the chassis under its falling center of mass. Because linear controllers cannot process non-linear trigonometric functions natively, the system must be rigorously linearized.

2 Hardware and System Architecture

Before mathematical modeling can begin, the physical parameters of the hardware must be defined. The physical system is constructed using the following core components:

- **Main Controller:** A Raspberry Pi 3 Model B, featuring a 1.4 GHz 64-bit quad-core processor, executing the control loop and sensor fusion algorithms in real-time.
- **Inertial Measurement Unit (IMU):** An MPU-9150 sensor combining a 3-axis gyroscope and a 3-axis accelerometer. It communicates with the processor via the I2C protocol to provide raw orientation data.
- **Actuators:** EMG30 DC gear motors equipped with built-in encoders for precise angular velocity feedback.
- **Motor Driver:** An MD-25 drive board, designed specifically for the EMG30, acting as the high-current interface between the logic-level controller and the motors.

2.1 System Parameters

The physical and electrical constants used for the mathematical derivations are summarized in Table 1.

Table 1: Physical and Electrical System Parameters

Symbol	Description	Typical Value / Unit
g	Gravitational acceleration	9.81 m/s^2
r	Wheel radius	0.05 m
M_W	Mass of a single wheel	0.13 kg
M_P	Mass of the pendulum (chassis)	1.24 kg
l	Length from wheel axle to chassis center of mass	0.213 m
I_W	Wheel's moment of inertia	0.0002899 kg $\cdot$ m^2
I_P	Chassis's moment of inertia	0.05626 kg $\cdot$ m^2
k_m	Motor torque constant	0.2774 V $\cdot$ m/A
k_e	Motor back EMF constant	0.509 V/(rad/s)
R	Equivalent resistance of motor armature	7.101 Ω

3 Dynamic System Modeling

The mathematical model is divided into the electromechanical actuator subsystem and the mechanical chassis subsystem.

3.1 Electromechanical Actuator (DC Motor) Model

The propulsion and restorative torques are generated by the DC gear motors. Applying Kirchhoff's voltage law to the armature circuit yields:

$$V_a = Ri + L\frac{di}{dt} + V_e \tag{1}$$

Where V_a is the applied voltage, and V_e is the back electromotive force (EMF), which opposes the applied voltage proportionally to the shaft's angular velocity (ω):

$$V_e = k_e\omega \tag{2}$$

The total torque (τ_{total}) generated is the difference between the electromagnetic torque ($i \cdot k_m$) and frictional losses (τ_f). By substituting the back EMF and neglecting negligible armature inductance ($L \approx 0$), the governing equation for the DC motor's angular acceleration becomes:

$$\dot{\omega} = \frac{k_m V_a}{I_W R} - \frac{k_m k_e \omega}{I_W R} - \frac{\tau_f}{I_W} \tag{3}$$

3.2 Mechanical Chassis Dynamics

By applying Newtonian free-body diagram analysis and Lagrangian energy methods to the wheels and the chassis, we can derive the governing equations of motion. Let θ_P represent the pitch angle (where π radians is perfectly vertical).

The dynamics are highly non-linear due to the coupling of linear displacement (x) and angular

displacement (θ_P). The rotational equation for the falling chassis is:

$$(I_P + l^2 M_P)\ddot{\theta}_P - \frac{2k_m k_e}{Rr}\dot{x} + \frac{2k_m}{R}V_a + M_P g l \sin\theta_P = -M_P l \ddot{x}\cos\theta_P \tag{4}$$

The translational equation for the base wheels is:

$$\frac{2k_m}{Rr}V_a = \left(2M_W + \frac{2I_W}{r^2} + M_P\right)\ddot{x} + \frac{2k_m k_e}{Rr^2}\dot{x} + M_P l\ddot{\theta}_P\cos\theta_P - M_P l\dot{\theta}_P^2\sin\theta_P \tag{5}$$

4 Linearization and State-Space Formulation

Because equations (4) and (5) contain non-linear trigonometric terms ($\sin\theta_P$, $\cos\theta_P$, and centripetal forces $\dot{\theta}_P^2$), a linear control system cannot be directly applied. We must linearize the system around the equilibrium operating point.

4.1 General Linearization Principle (Taylor Series)

The mathematical foundation for linearizing a non-linear function $f(x)$ around a specific operating point x_0 relies on truncating the Taylor series expansion to its first-order linear terms:

$$f(x) \approx f(x_0) + \left.\frac{df}{dx}\right|_{x=x_0}(x - x_0) \tag{6}$$

For complex multi-variable systems where the state derivatives are non-linear functions of the states and inputs, $\dot{X} = f(X, U)$, this principle is expanded using Jacobian matrices. The system is linearly approximated as $\dot{X} = AX + BU$, where the system matrix A and input matrix B are defined by the partial derivatives evaluated at the equilibrium state (X_e) and input (U_e):

$$A = \left[\frac{\partial f}{\partial X}\right]_{X_e,U_e}, \quad B = \left[\frac{\partial f}{\partial U}\right]_{X_e,U_e} \tag{7}$$

4.2 Small-Angle Approximations Application

The target equilibrium point is standing perfectly upright. We define a small deviation angle φ such that the actual position is $\theta_P = \pi + \varphi$. Applying the aforementioned Taylor series principle to the trigonometric functions near zero deviation ($\varphi \approx 0$), we evaluate the following specific approximations:

$$\cos(\pi + \varphi) \approx -1 \tag{8}$$

$$\sin(\pi + \varphi) \approx -\varphi \tag{9}$$

$$(\dot{\theta}_P)^2 = (\dot{\varphi})^2 \approx 0 \tag{10}$$

4.3 State-Space Representation

By substituting these limits back into the mechanical equations, the trigonometric complexities drop out, yielding purely linear differential equations. We define our state vector X as:

$$X = \begin{bmatrix} x \\ \dot{x} \\ \varphi \\ \dot{\varphi} \end{bmatrix} = \begin{bmatrix} \text{Position} \\ \text{Linear Velocity} \\ \text{Tilt Angle} \\ \text{Angular Velocity} \end{bmatrix} \tag{11}$$

The generalized continuous-time state-space model is expressed as:

$$\dot{X} = AX + BU, \quad Y = CX + DU \tag{12}$$

Where U is the control input (V_a). By isolating the highest-order derivatives ($\ddot{x}$ and $\ddot{\varphi}$) from the linearized equations, we populate the system matrix (A) and input matrix (B). To simplify the notation, we define two grouping constants based on the mechanical properties:

$$\beta = 2M_W + \frac{2I_W}{r^2} + M_P$$
$$\alpha = I_P\beta + M_P l^2 \left(2M_W + \frac{2I_W}{r^2}\right)$$

The state-space matrices formulated analytically are explicitly defined as:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -\frac{2k_m k_e(M_P lr + I_P + M_P l^2)}{Rr^2\alpha} & \frac{M_P^2 g l^2}{\alpha} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{2k_m k_e(r\beta - M_P l)}{Rr^2\alpha} & \frac{M_P g l \beta}{\alpha} & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ \frac{2k_m(I_P + M_P l^2 + M_P lr)}{Rr\alpha} \\ 0 \\ \frac{2k_m(M_P l - r\beta)}{Rr\alpha} \end{bmatrix} \tag{13}$$

Because the output we wish to observe is the tilt angle (φ), the observation matrix C is set to extract the third state variable, and the feedforward matrix D is 0:

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \end{bmatrix}, \quad D = [0] \tag{14}$$

5 Sensor Fusion and State Estimation (Kalman Filter)

To utilize the control model, the microcontroller requires highly accurate feedback of the state φ. The MPU-9150 IMU provides raw accelerometer and gyroscope data. However, accelerometers are highly sensitive to chassis vibration, and gyroscopes suffer from temporal drift.

To mitigate this, a Kalman filter is deployed. As a recursive time-domain filter, it optimally estimates the true angle by minimizing the mean square error between the predicted state and the noisy measurement.

1. Time Update (Predict Step):

$$\hat{x}_k^- = A\hat{x}_{k-1} + Bu_k \tag{15}$$
$$P_k^- = AP_{k-1}A^T + Q \tag{16}$$

Where $\hat{x}_k^-$ is the a priori state estimate, P is the error covariance, and Q is the process noise covariance.

2. Measurement Update (Correct Step):

$$K_k = P_k^- H^T (HP_k^- H^T + R)^{-1} \tag{17}$$
$$\hat{x}_k = \hat{x}_k^- + K_k(z_k - H\hat{x}_k^-) \tag{18}$$
$$P_k = (I - K_k H)P_k^- \tag{19}$$

Where K_k is the optimal Kalman gain, z_k is the actual sensor measurement, and R is the measurement noise covariance.

This filter guarantees that the PID controller is fed a pristine, drift-free angular displacement signal.

6 Control Strategy and Simulation Results

With the system rigorously linearized and sensor noise attenuated, a continuous-time Proportional-Integral-Derivative (PID) controller is applied to the tilt angle error $e(t) = \varphi_{target} - \varphi_{actual}$, where $\varphi_{target} = 0$.

6.1 Numerical State-Space and Transfer Function

Using the derived formulas and the system parameters, the state-space system was computed in MATLAB. The resulting numerical matrices for the plant dynamics are:

$$A = \begin{bmatrix} 0 & 1.0000 & 0 & 0 \\ 0 & -14.1970 & 23.2848 & 0 \\ 0 & 0 & 0 & 1.0000 \\ 0 & -25.3778 & 40.3275 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1.3946 \\ 0 \\ 2.4929 \end{bmatrix} \tag{20}$$

Based on these matrices, the open-loop transfer function of the plant was extracted. The transfer function features a numerator of $[0,\ 0,\ 2.4929,\ -6.0971\times10^{-16}]$ and a denominator of $[1,\ 14.1970,\ -40.3275,\ 18.388$

6.2 PID Control and Tuning Methodology

The control law determining the voltage applied to the motors is:

$$V_a(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt} \tag{21}$$

Through analytical tuning in MATLAB, the optimal PID gains were determined:

- **Proportional Gain ($K_p = 194.534$):** Provides the primary restorative force. A high K_p ensures the robot reacts quickly to falling, acting as a stiff virtual spring.

- **Integral Gain ($K_i = 1432.586$):** Addresses steady-state errors. If the physical chassis has an imperfect center of gravity, it will naturally lean slightly to one side. The integral term accumulates this tiny sustained error and provides a permanent voltage offset.

- **Derivative Gain ($K_d = 5.551$):** Acts as a virtual damper. By analyzing the angular velocity, the controller anticipates where the robot will be, preventing the robot from violently overshooting the vertical setpoint.

6.3 Simulink Block Diagram Setup

To simulate realistic behavior and visualize continuous control, a Simulink model was constructed. The architecture of the control loop consists of:

- **Setpoint Balance:** A constant block set to 0, representing the desired vertical angle.

- **Summation Block:** Compares the desired setpoint with the actual angle, calculating the current error signal.

- **PID Controller Block:** Applies the calculated continuous-time gains to generate the control effort.

- **Saturation Block:** Restricts the PID output voltage to match the physical limits of the DC motors (e.g., between -12V and +12V), preventing impossible torque demands.

- **State-Space Plant:** Contains the A, B, C, D matrices, representing the physical dynamics of the chassis and wheels.

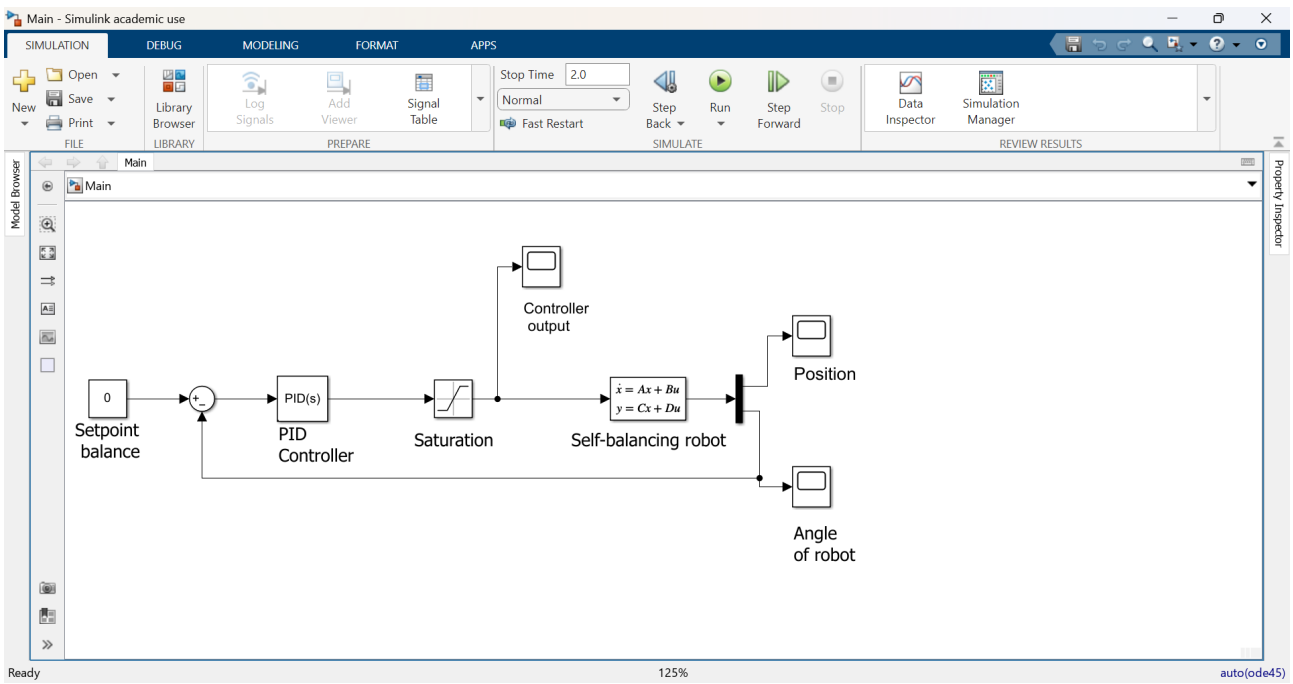

Figure 1: Simulink Closed-Loop Control Block Diagram

6.4 Step Response and Scope Analysis

When the loop is closed using the designed PID controller, the system stabilizes successfully. The step response information generated by MATLAB showcases excellent performance metrics:

- **Rise Time:** 0.0571 seconds
- **Settling Time:** 0.4881 seconds
- **Transient Time:** 0.4881 seconds
- **Overshoot:** 41.5260%
- **Peak Value:** 1.4080 (at Peak Time: 0.1540 seconds)

The simulation over a 2.0-second period reveals three primary behavioral traits:

1. **Robot Tilt Angle:** The angle experiences a sharp initial deviation but is rapidly pushed back towards 0 radians by the controller. It settles completely by 0.5 seconds.

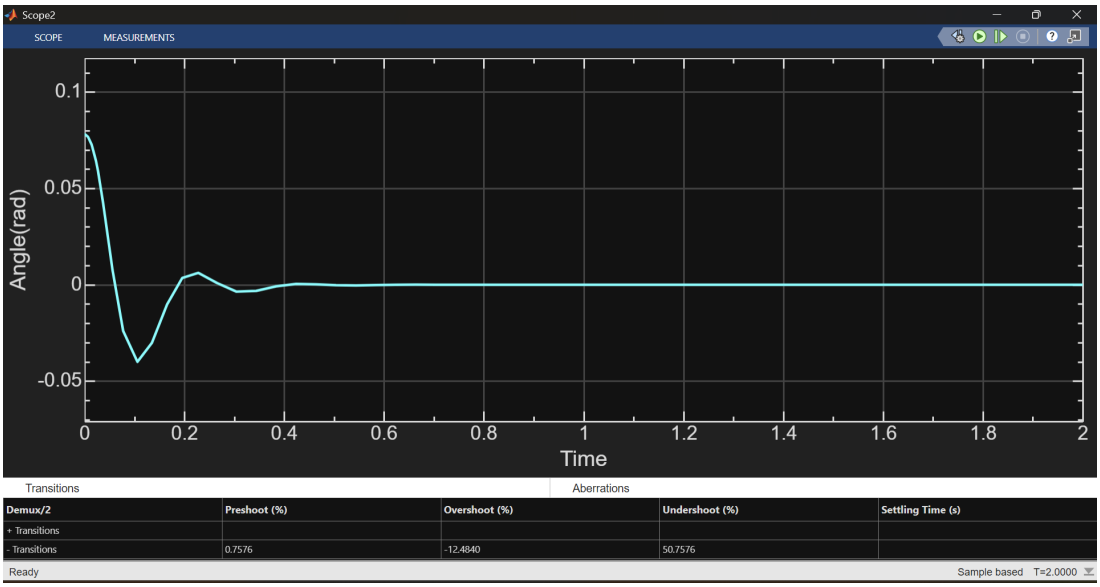

Figure 2: Robot Tilt Angle (θ) Response over time, demonstrating correction to steady-state equilibrium.

2. **Robot Distance (Position):** To counteract the falling motion, the wheels drive forward, peaking around 0.1 seconds (drifting to approximately -0.05 m) before stabilizing at a constant offset.

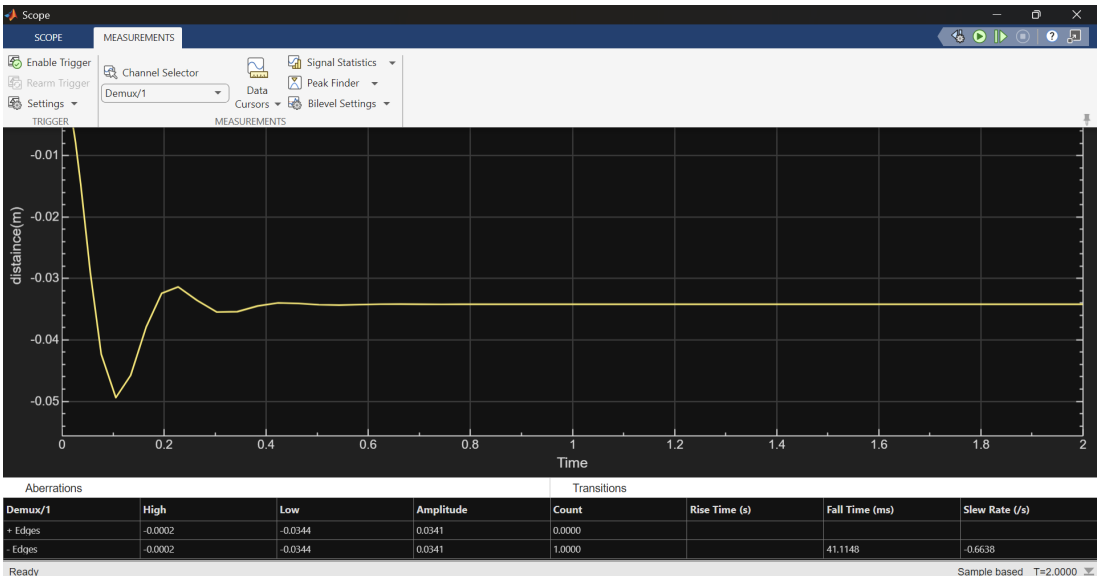

Figure 3: Robot Distance (Position, x) Tracking Response over time.

3. **PID Output (Control Signal):** The control effort demonstrates a massive initial spike to combat the falling angle, limited by the saturation blocks. As the robot approaches the upright position, the voltage required smoothly tapers off back to zero.

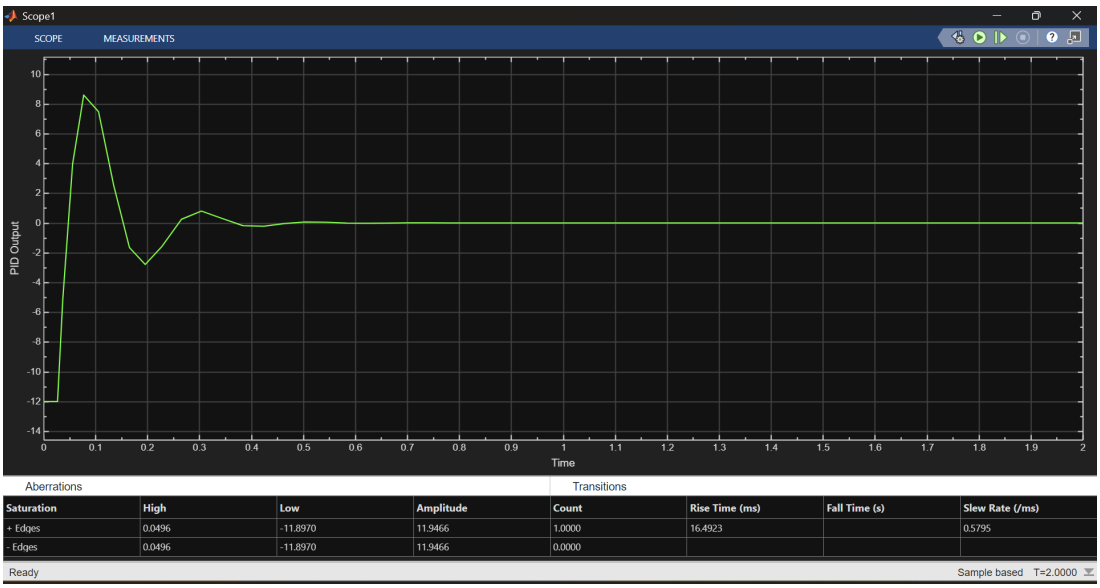

Figure 4: PID Output Control Signal Effort (Motor Voltage).

To validate the numerical data visually, a realistic MATLAB animation was scripted to plot the wheels and chassis dynamically over a 15-second period, accurately demonstrating the robot moving forward, backward, and tracking balance trajectories.

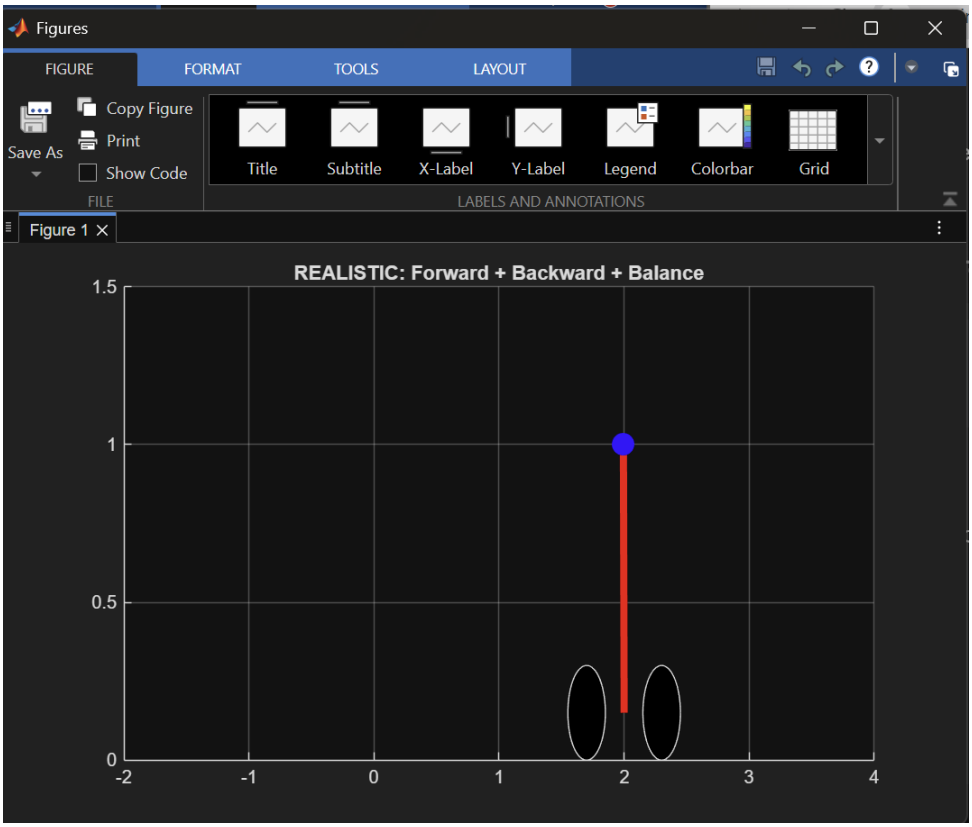

Figure 5: Realistic MATLAB Animation snapshot verifying Forward, Backward, and Balance movement based on the control logic.

7 Conclusion

The stabilization of a two-wheeled inverted pendulum is impossible through empirical guessing; it demands stringent mathematical physics. This report successfully modeled the electrical properties of the DC drive motors and the Lagrangian dynamics of the mechanical chassis. By applying Taylor series limits, the non-linear gravitational vectors were mathematically linearized around the vertical equilibrium point, allowing the extraction of the numerical A and B state-space matrices.

Coupled with a Kalman filter for optimal sensor state estimation, this theoretical foundation proved the efficacy of utilizing a tuned PID controller ($K_p = 194.534$, $K_i = 1432.586$, $K_d = 5.551$). MATLAB and Simulink validations confirmed that the closed-loop system successfully manages initial overshoot, rectifies sudden tilt angles, and brings the inherently unstable plant back to steady-state equilibrium in less than 0.5 seconds. The inclusion of a 3D rigid body simulation in CoppeliaSim further verified the control logic's viability in a physical environment, successfully overcoming script and dynamic joint errors to achieve balance.

8 Future Scope

While classical integer-order PID control achieves excellent baseline stability, future iterations of this project will explore more advanced architectures:

1. **Fractional-Order PID (FOPID):** Expanding the controller to $PI^{\lambda}D^{\mu}$. By utilizing fractional calculus (where the integral and derivative orders are non-integers, e.g., $\lambda = 1.35, \mu = 0.80$), the controller gains two additional degrees of freedom, potentially reducing overshoot by up to 10% and improving rise time.

2. **Linear Quadratic Regulator (LQR):** Utilizing the full state-space A and B matrices derived in Section 4 to implement an optimal modern state-feedback controller. This would allow the simultaneous, mathematically optimal control of both the tilt angle *and* the linear displacement across the floor.

3. **Hardware Modeling and Real-World Deployment:** Transitioning the verified mathematical model from the simulation environment to a physical prototype. This involves comprehensive 3D CAD modeling of the chassis to optimize the center of mass, accurate actuator selection, and the implementation of the control algorithms onto an embedded microcontroller (such as a Raspberry Pi) utilizing real-time digital IIR filters for robust sensor fusion.

9 3D Rigid Body Simulation in CoppeliaSim

To bridge the gap between mathematical simulation in MATLAB and real-world implementation, a 3D physical model of the self-balancing robot was constructed and tested in the CoppeliaSim (formerly V-REP) environment. This provided a realistic testbed for examining rigid body physics, collision dynamics, and joint friction.

9.1 Model Construction and Hierarchy

The robot's physical structure was assembled using primitive 3D shapes representing the main chassis, wheels, and motors. A clear parent-child hierarchy was established in the scene to accurately replicate physical dependencies. As shown in Figure 7, the `CHASIS` acts as the parent object, connected to `LEFT_MOTOR` and `RIGHT_MOTOR` joints, which in turn drive the respective wheels. A simulated IMU was also attached to the chassis to feed orientation data to the control script.

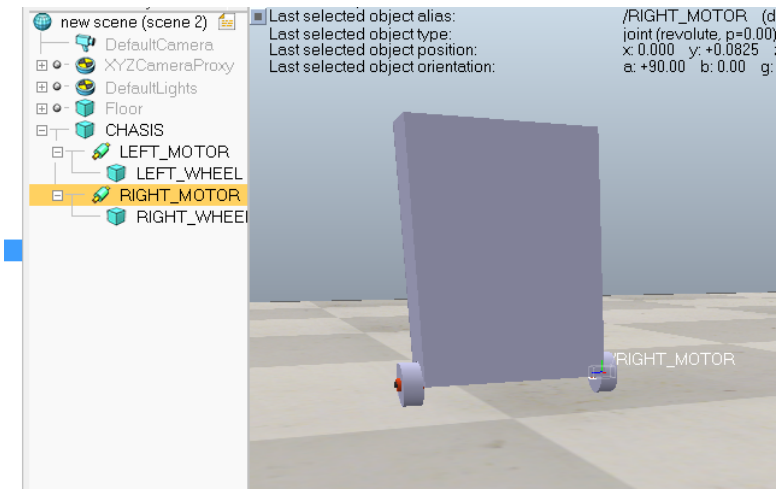

Figure 6: Initial setup of the robot body and wheels in CoppeliaSim workspace.

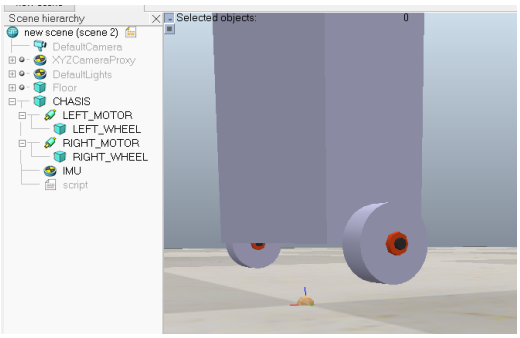

Figure 7: Scene hierarchy establishing the physical relationship between the chassis, motor joints, and wheels.

9.2 Troubleshooting and Dynamic Properties

Transitioning from a pure mathematical model to a rigid body physics engine naturally introduced simulation challenges. Initial tests produced dynamic property warnings, specifically indicating that the `/left_joint` was not dynamically enabled (Figure 8), preventing the physics engine from computing realistic torque transfers.

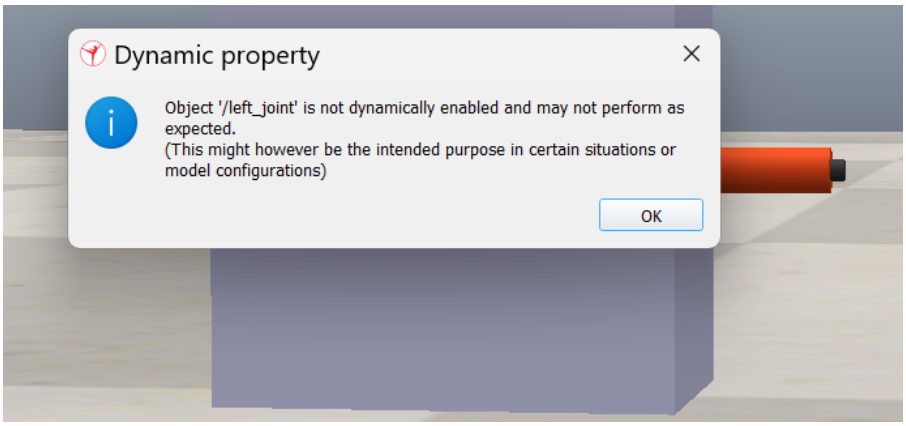

Figure 8: Dynamic property warning indicating issues with joint physics configuration.

Furthermore, during the execution of the embedded control loop, several Lua scripting errors arose. Errors such as `Exception in sim.getObject` indicated improperly formatted aliases or paths in the script, while `AttributeError` exceptions pointed to missing static attributes during the cleanup phase (Figure 9).

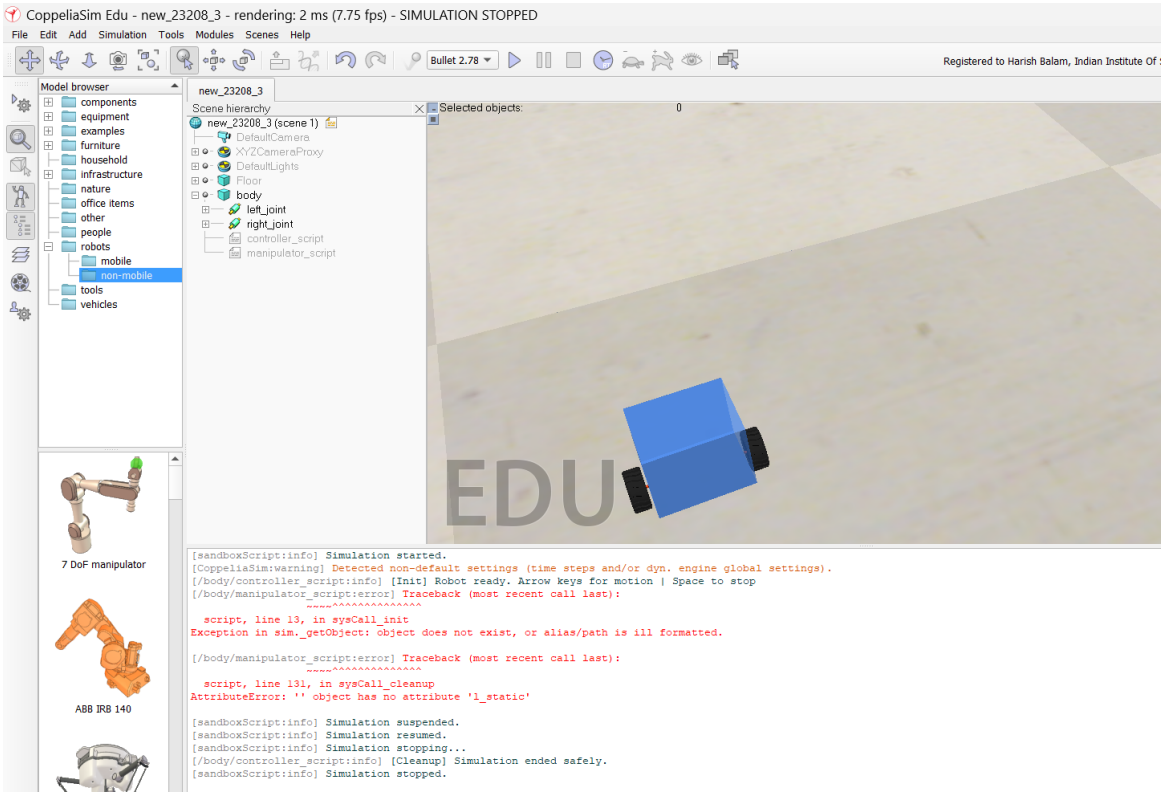

Figure 9: CoppeliaSim console displaying Lua script execution errors during the initialization phase.

9.3 Simulation Resolution and Balancing Performance

Following rigorous debugging, the object path formatting errors in the Lua scripts were successfully resolved. The motor joints were reconfigured to be dynamically enabled in the physics engine, allowing torque to be successfully applied based on the PID calculations.

As a result, the simulation successfully executed, and the robot demonstrated the ability to maintain partial balance in the 3D environment. Figure 10 shows the robot successfully exported and running without fatal errors, proving that the calculated control logic holds true even when subjected to simulated physical constraints.

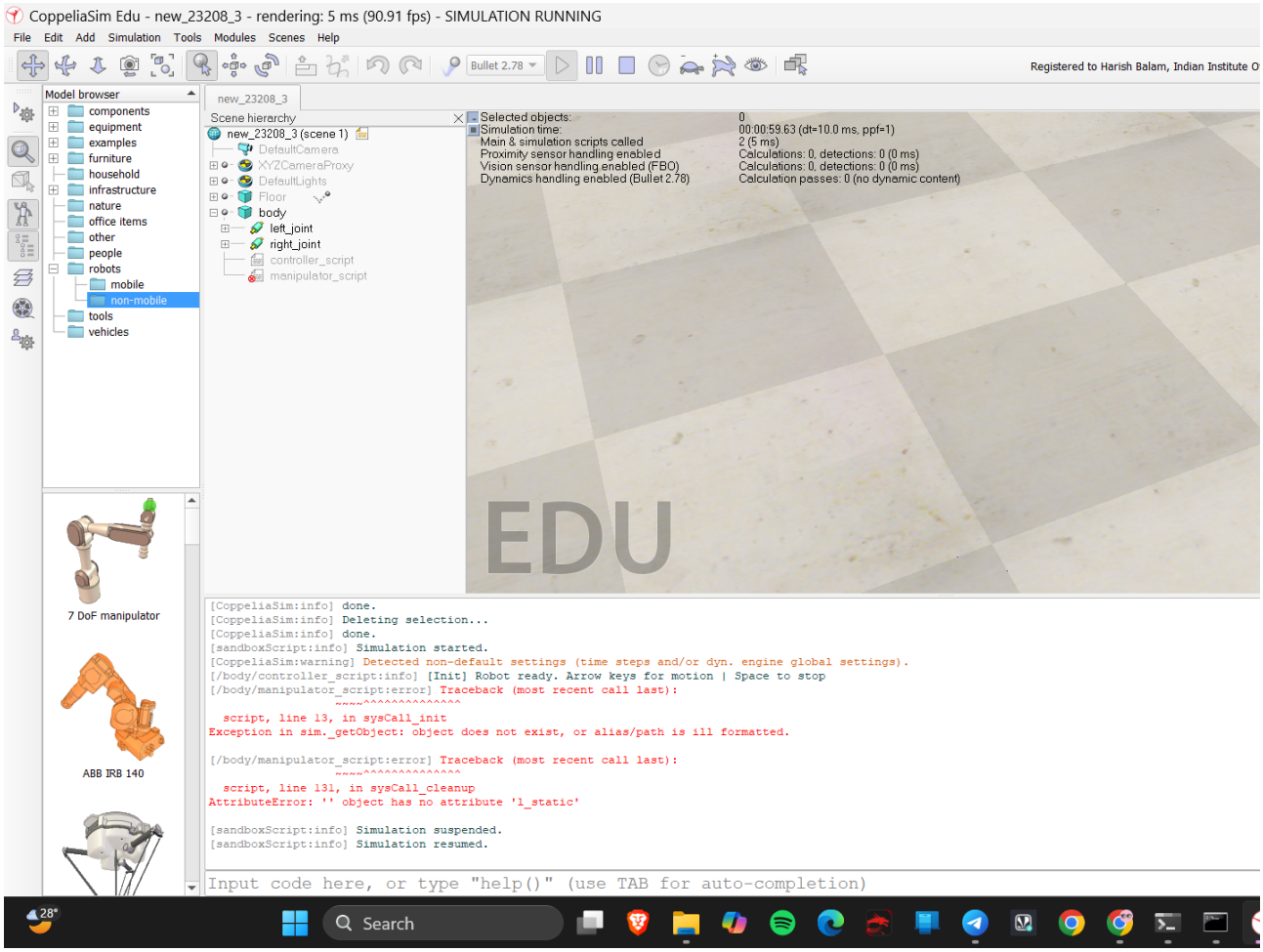

Figure 10: Final stable execution of the self-balancing simulation in CoppeliaSim.

A MATLAB Source Code

The following MATLAB scripts were authored to calculate system matrices, simulate the step response, and animate the physical system dynamically as referenced in the Simulation Results section.

A.1 System Calculation and Step Response Script

```
clc; clear; close all;

% System Parameters
g = 9.81;
r = 0.05;
MW = 0.13;
MP = 1.24;
l = 0.213;
IW = 0.0002899;
IP = 0.05626;
km = 0.2774;
ke = 0.509;
R = 7.101;

% Intermediate variables
beta = 2*MW + 2*IW/(r^2) + MP;
alpha = IP*beta + MP*l^2*(MW + IW/(r^2));

% State Space Matrices
A = [0 1 0 0;
     0 (2*km*ke*(MP*l*r + IP + MP*l^2))/(R*r^2*alpha) (MP^2*g*l^2)/alpha 0;
     0 0 0 1;
     0 (2*km*ke*(r*beta - MP*l))/(R*r^2*alpha) (MP*g*l*beta)/alpha 0];

B = [0;
     (2*km*(IP + MP*l^2 + MP*l*r))/(R*r*alpha);
     0;
     (2*km*(MP*l - r*beta))/(R*r*alpha)];

C = [0 0 1 0];
D = 0;

sys = ss(A,B,C,D);
G = tf(sys);

% PID Controller
Kp = 194.534;
Ki = 1432.586;
Kd = 5.551;

PID = pid(Kp,Ki,Kd);

closed_sys = feedback(PID*G, 1);
t = 0:0.001:2;
[y, t] = step(closed_sys, t);
```

```
info = stepinfo(y, t);

% Output Display
disp('A Matrix:');
disp(A)
disp('B Matrix:');
disp(B)
disp('C Matrix:');
disp(C)
disp('D Matrix:');
disp(D)
disp('Transfer Function:');
disp(G)
disp('PID Gains:')
disp(['Kp = ', num2str(Kp)])
disp(['Ki = ', num2str(Ki)])
disp(['Kd = ', num2str(Kd)])
disp('Performance:')
disp(info)
```

A.2 Robot Animation Script

```
clc; close all;

t = 0:0.05:15;
theta = zeros(size(t));
x_move = zeros(size(t));

% Trajectory logic
for i = 1:length(t)
    if t(i) < 4
        theta(i) = 0.15;
        x_move(i) = 0.5*t(i);
    elseif t(i) < 8
        theta(i) = 0.15*exp(-0.5*(t(i)-4)).*cos(3*(t(i)-4));
        x_move(i) = 2 + 0.3*(t(i)-4);
    elseif t(i) < 11
        theta(i) = -0.15;
        x_move(i) = 3.2 - 0.4*(t(i)-8);
    else
        theta(i) = 0.15*exp(-0.8*(t(i)-11)).*cos(4*(t(i)-11));
        x_move(i) = 2;
    end
end

L = 1;
wheel_r = 0.15;
wheel_distance = 0.6;

figure

```

```
% Animation rendering
for i = 1:length(t)
    clf
    w1 = x_move(i) - wheel_distance/2;
    w2 = x_move(i) + wheel_distance/2;

    xb = x_move(i) + [0 L*sin(theta(i))];
    yb = [wheel_r L*cos(theta(i))];

    rectangle('Position', [w1-wheel_r, 0, 2*wheel_r, 2*wheel_r], 'Curvature', [1 1],
        'FaceColor', 'k')
    hold on
    rectangle('Position', [w2-wheel_r, 0, 2*wheel_r, 2*wheel_r], 'Curvature', [1 1],
        'FaceColor', 'k')

    plot(xb, yb, 'r', 'LineWidth', 4)
    plot(xb(2), yb(2), 'bo', 'MarkerSize', 12, 'MarkerFaceColor', 'b')

    axis([-2 4 0 1.5])
    grid on
    title('REALISTIC: Forward+Backward+Balance')
    drawnow
end
```